



Green University of Bangladesh

Department of Computer Science and Engineering (CSE)

Semester: Summer, Year: 2026, B.Sc. in CSE (Day)

CIPHERLINK: Design and Evaluation of a Secure Real-Time Peer-to-Peer Encrypted Communication Network

Course Title: Cryptography and Cybersecurity

Course Code: CSE 323

Section: 231 D2

Students Details

Name	ID
Ahmad Jamil	231902007
Asraful Hasan	231902011
Md Shahadat Hossain	231902012

Submission Date: 04/08/2026

Course Teacher's Name: Sharifur Rahman

[For teachers use only: **Don't write anything inside this box**]

<u>Lab Project Status</u>	
Marks:	Signature:
Comments:	Date:

Abstract

CIPHERLINK is a prototype real-time communication system that combines authenticated peer discovery, WebSocket-based signaling, elliptic-curve key agreement, and authenticated symmetric encryption. The project provides both browser and command-line interfaces through which two users can create identities, discover peers, establish a secure session, and exchange messages without persistent server-side message storage. Its cryptographic design uses ephemeral Elliptic Curve Diffie–Hellman (ECDH) over the NIST P-384 curve, HKDF-SHA-256 for key derivation, and AES-256-GCM for authenticated encryption. This report documents the motivation, requirements, architecture, implementation workflow, encryption and decryption algorithms, threat model, and evaluation procedure described by the project documentation. The analysis finds that the architecture provides a reasonable prototype basis for confidentiality, integrity, replay resistance, and forward secrecy when implemented correctly. It also identifies an important limitation: P-384 ECDH is strong against known classical attacks but is not post-quantum secure because a sufficiently capable fault-tolerant quantum computer could apply Shor’s algorithm. Therefore, “quantum-resistant” is treated as a project goal and product label rather than a demonstrated property. Future work should introduce a standardized post-quantum or hybrid key-establishment mechanism, formalize the wire protocol, and validate the implementation through independent security testing.

Acknowledgment

The authors acknowledge the guidance of the course instructor and the support of the Department of Computer Science and Engineering. The report was prepared from the CIPHERLINK Version 1.0 user manual and technical documentation supplied with the project.

Contents

Abstract	i
Acknowledgment	i
1 Introduction	1
1.1 Overview	1
1.1.1 Core capabilities	1
1.2 Motivation	2
1.2.1 Security motivation	2
1.2.2 Usability motivation	2
1.3 Problem Definition	2
1.3.1 Problem statement	2
1.3.2 Functional requirements	2
1.3.3 Non-functional requirements	3
1.3.4 Complex engineering problem	3
1.4 Design Goals and Objectives	4
1.4.1 Primary objectives	4
1.4.2 Security objectives	4
1.4.3 Scope boundary	4
1.5 Applications	4
1.5.1 Private collaboration	4
1.5.2 Security education and experimentation	4
1.5.3 Prototype integration	5
2 Design, Development, and Implementation	7
2.1 System Architecture	7
2.1.1 Backend	7
2.1.2 Frontend and CLI	7

2.2	Trust Boundaries and Data Flow	8
2.2.1	Authentication and discovery	8
2.2.2	Key establishment	8
2.2.3	Message protection	8
2.3	User Interface Design	9
2.3.1	Identity and Nodes views	9
2.3.2	Session and Vault views	9
2.3.3	Visual system	9
2.4	Implementation Procedure	10
2.4.1	Environment and startup	10
2.4.2	Connection state machine	10
2.4.3	WebSocket message structure	10
2.5	Algorithms	11
2.6	Security Design Analysis	12
2.6.1	Eavesdropping and tampering	12
2.6.2	Man-in-the-middle attacks	12
2.6.3	Replay and key compromise	12
2.6.4	Quantum-security qualification	12
3	Performance Evaluation	13
3.1	Evaluation Environment and Procedure	13
3.1.1	Documented environment	13
3.1.2	Functional test procedure	13
3.1.3	Performance measurement procedure	14
3.2	Results Analysis and Testing	14
3.2.1	Functional results from project documentation	14
3.2.2	Reported performance targets	14
3.2.3	Threat-resistance results	15
3.3	Overall Discussion	15
3.3.1	Complex engineering problem discussion	16
4	Conclusion	17
4.1	Discussion	17
4.2	Limitations	17
4.3	Scope of Future Work	17

Chapter 1

Introduction

1.1 Overview

Modern messaging systems carry private academic, personal, and organizational information across networks that may be monitored or modified by an adversary. Transport encryption protects a connection to a service provider, but it does not by itself guarantee that the provider cannot access message content. End-to-end encryption (E2EE) addresses this trust problem by encrypting data at the sender and decrypting it only at the intended recipient.

CIPHERLINK is a prototype secure communication network developed during a 36-hour security hackathon. It offers a web interface and a command-line client for real-time peer communication. A coordinating server authenticates users, maintains peer presence, and facilitates public-key exchange. After session establishment, message confidentiality and integrity are provided at the endpoints using ECDH, HKDF, and AES-256-GCM. The design also avoids persistent server-side storage of plaintext messages [1–3].

1.1.1 Core capabilities

The system is designed to provide the following capabilities:

- identity creation and authentication using elliptic-curve keys and certificates;
- discovery of available peers and authenticated session establishment;
- real-time messaging through WebSocket signaling and a peer-oriented communication flow;
- confidentiality and integrity through AES-256-GCM;
- fresh session secrets through ephemeral ECDH and HKDF-SHA-256;
- browser and command-line interaction modes; and
- volatile message handling so that the server is not a message archive.

1.2 Motivation

Centralized communication platforms create attractive targets because they aggregate identity data, message history, and metadata. A server breach, malicious administrator, weak access control, or misconfigured database may expose a large collection of conversations. CIPHERLINK explores a reduced-trust model in which the server coordinates participants but endpoints retain cryptographic control of readable content.

1.2.1 Security motivation

The project focuses on four practical security goals. First, a passive observer should not learn the plaintext from captured traffic. Second, an active attacker should not be able to modify a message without detection. Third, compromise of a long-term identity key should not automatically reveal previously completed sessions. Fourth, messages should not remain in a server database after delivery. These goals motivate the use of authenticated encryption, ephemeral key exchange, and limited retention [3, 4].

1.2.2 Usability motivation

Strong cryptography is useful only when users can operate it correctly. CIPHERLINK therefore exposes complex operations through familiar actions: create an identity, discover a node, select *Secure Link*, and send a payload. Status colors, verification marks, connection banners, and cipher-stream indicators communicate the current security state. A CLI is also provided for testing, automation, and users who prefer terminal workflows [1].

1.3 Problem Definition

1.3.1 Problem statement

The project addresses the problem of enabling two mutually identified users to exchange real-time messages over an untrusted network while minimizing the coordinating server's access to message content. The solution must securely agree on keys, protect messages against disclosure and alteration, identify stale or repeated traffic, provide responsive user feedback, and remain feasible within a Python and browser-based prototype.

1.3.2 Functional requirements

1. A user shall be able to generate or load an identity and register it with the server.
2. The server shall expose available peers and mediate connection requests.
3. Two accepted peers shall derive compatible session keys without transmitting the shared secret.

4. The sender shall encrypt each message before transmission.
5. The receiver shall reject modified, unauthenticated, or duplicate messages.
6. The application shall display connection and verification status in real time.
7. The application shall support both a web UI and a CLI.

1.3.3 Non-functional requirements

The prototype should provide low interactive latency, tolerate temporary WebSocket disconnection through reconnection logic, use a cryptographically secure random number generator, avoid persistent message storage, and keep the interface understandable on desktop and mobile displays. Security-sensitive failures must be closed: certificate, tag, key-confirmation, and replay checks must stop message processing rather than silently continue.

1.3.4 Complex engineering problem

Table 1.1 relates the project to common complex-engineering attributes.

Table 1.1: Complex engineering attributes addressed by CIPHERLINK

Attribute	How the project addresses it
P1: Depth of knowledge required	Requires applied cryptography, asynchronous Python, WebSockets, TLS, browser programming, key management, and secure protocol design.
P2: Conflicting requirements	Strong authentication and key rotation increase security but may reduce usability, interoperability, and performance. Volatile storage improves privacy but removes message recovery.
P3: Depth of analysis required	Correctness depends on analyzing trust boundaries, nonce uniqueness, key separation, certificate validation, replay state, failure behavior, and attack paths.
P4: Familiarity of issues	Messaging is familiar, but composing cryptographic primitives and communicating accurate security guarantees involve specialized and partly unfamiliar issues.
P5: Extent of applicable codes	The design draws on NIST recommendations for key establishment and GCM, HKDF specifications, TLS, certificate practice, and WebSocket behavior.
P6: Stakeholder involvement	End users value simplicity and privacy; administrators value observability and availability; security reviewers require verifiable protocol behavior. These needs may conflict.
P7: Interdependence	Authentication, signaling, key exchange, encryption, UI state, and network availability are tightly coupled; failure in one component can invalidate the secure session.

1.4 Design Goals and Objectives

1.4.1 Primary objectives

The primary objective is to demonstrate a complete secure-messaging lifecycle rather than an isolated encryption function. The system must establish identity, negotiate a secret, derive independent key material, encrypt data, verify the received data, and expose the outcome to the user. A further objective is crypto-agility: algorithms and protocol fields should be replaceable so that a future version can adopt post-quantum key establishment.

1.4.2 Security objectives

The intended properties are confidentiality, integrity, endpoint authentication, session freshness, forward secrecy, and replay detection. AES-GCM is an authenticated-encryption mode standardized by NIST [3]. HKDF expands a high-entropy shared secret into context-specific keys, reducing the risk of reusing one key for incompatible purposes [4]. Ephemeral ECDH supports forward secrecy when private ephemeral values are securely erased after use [2].

1.4.3 Scope boundary

The project is a prototype, not a formally verified production messenger. It does not claim anonymity, protection from a compromised endpoint, guaranteed metadata concealment, or long-term post-quantum confidentiality. The server remains trusted for availability, identity registration, and correct signaling unless stronger end-to-end identity verification is added.

1.5 Applications

1.5.1 Private collaboration

CIPHERLINK can support short-lived, sensitive discussions among research partners, development teams, or incident-response personnel when participants require rapid communication and do not want a central message archive.

1.5.2 Security education and experimentation

The system is particularly suitable as a teaching platform. Students can observe key generation, public-key exchange, ciphertext transmission, authentication failure, replay detection, and session re-keying through both graphical and terminal interfaces.

1.5.3 Prototype integration

The modular Python backend and JSON/WebSocket message format can serve as a basis for secure device coordination, protected administrative channels, or embedded collaboration features. Production use would first require a protocol audit, hardened key storage, robust account recovery, scalable peer routing, and post-quantum migration.

Figures 1.1–1.4 present the main CIPHERLINK prototype interfaces. They illustrate the complete user flow from discovering available peers and managing encrypted payloads to creating an identity and establishing an encrypted communication session.

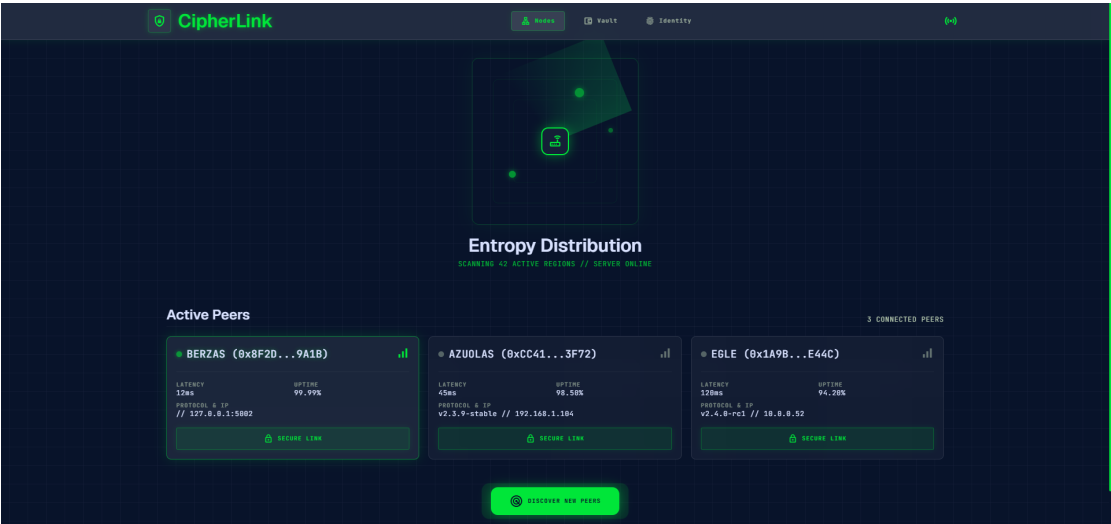


Figure 1.1: SS1: Active peer discovery and secure-link interface

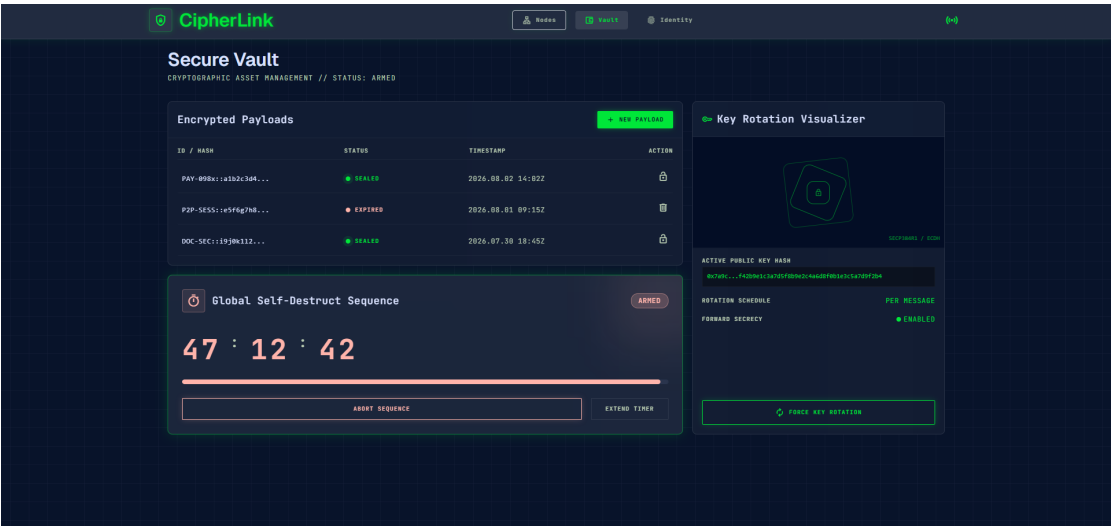
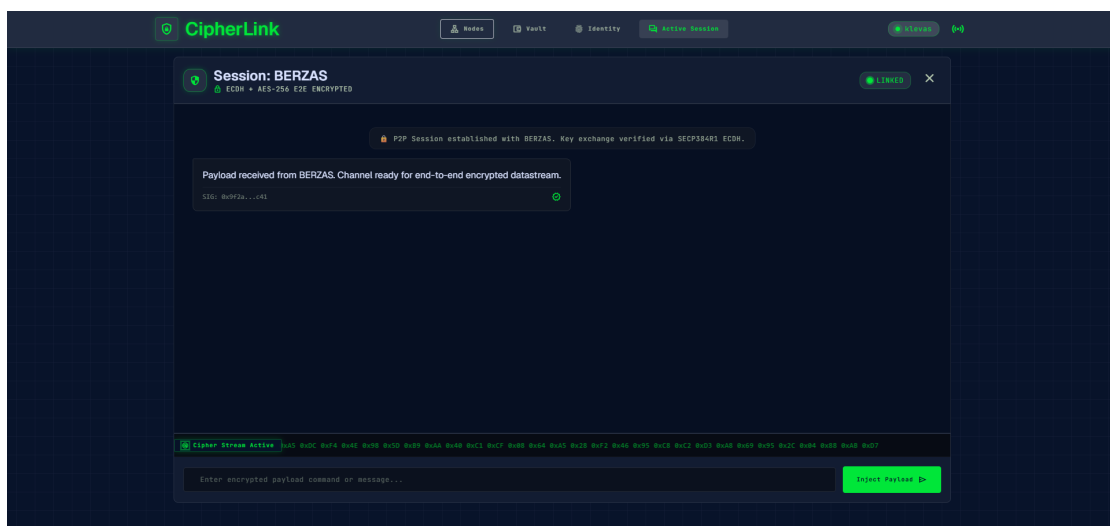
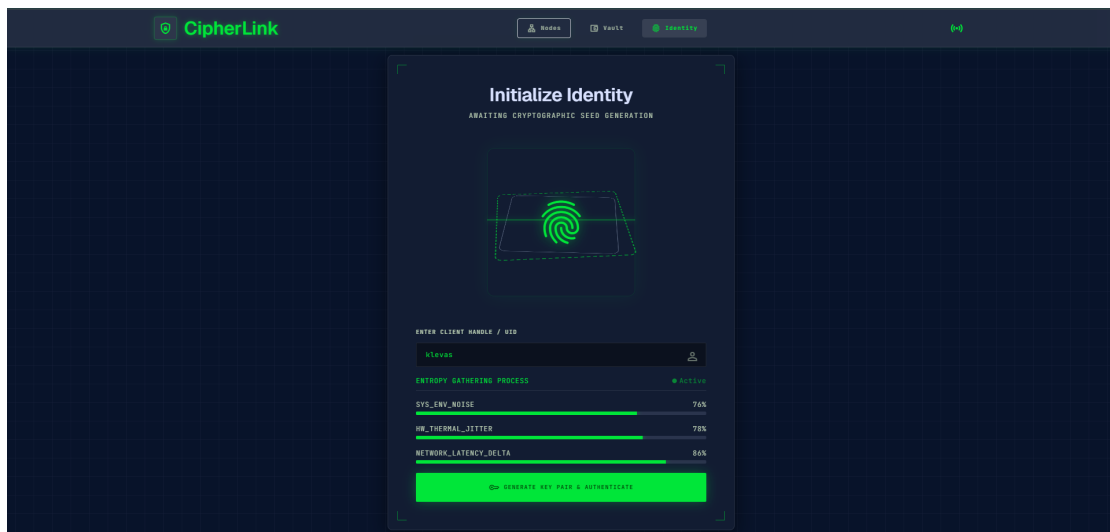


Figure 1.2: SS2: Secure Vault, self-destruct controls, and key-rotation interface



Chapter 2

Design, Development, and Implementation

2.1 System Architecture

The architecture separates coordination from content protection. The server hosts HTTP resources, authenticates clients, maintains a peer directory, and relays signaling data. Each client owns its private key material and performs session key derivation, encryption, authentication, and decryption locally. Figure 2.1 shows this separation.

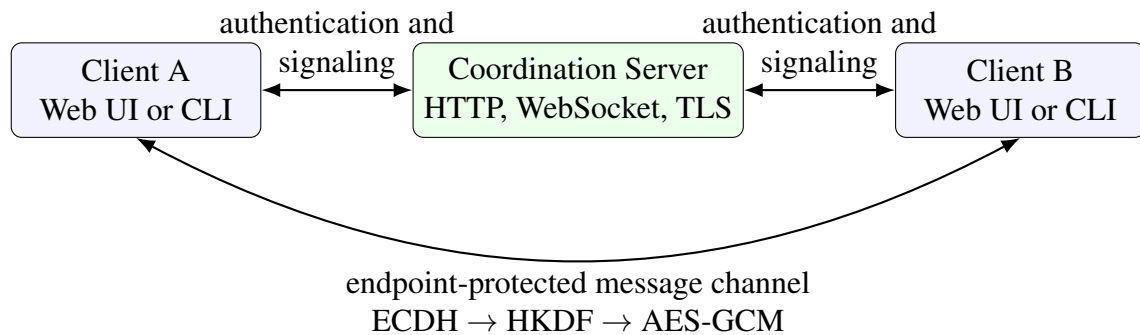


Figure 2.1: High-level CIPHERLINK architecture

2.1.1 Backend

The backend is implemented in Python using `aiohttp` for asynchronous HTTP and WebSocket services. The cryptography package supplies elliptic-curve, key-derivation, hashing, and authenticated-encryption operations. Python’s TLS support protects client–server transport and supports certificate-based authentication [1, 5].

2.1.2 Frontend and CLI

The browser client uses HTML5, Tailwind-based CSS, vanilla JavaScript, and the WebSocket API. Major views are Nodes, Vault, Identity, and Session. The CLI exposes the

same basic lifecycle with textual prompts and optional verbose cryptographic metadata. Supporting both interfaces separates protocol behavior from presentation and improves testability.

2.2 Trust Boundaries and Data Flow

2.2.1 Authentication and discovery

A client first creates or loads an identity and authenticates to the coordination service. The server publishes presence information to eligible users. A connection initiator selects a peer and sends a request; the peer accepts or rejects it. Certificate validation and identity-to-key binding are essential because encryption without authenticated keys remains vulnerable to a man-in-the-middle attack.

2.2.2 Key establishment

Each endpoint generates an ephemeral P-384 ECDH key pair. Public keys are exchanged through the authenticated signaling path. Client *A* computes

$$Z_A = \text{ECDH}(d_A, Q_B), \quad (2.1)$$

while client *B* computes

$$Z_B = \text{ECDH}(d_B, Q_A). \quad (2.2)$$

For valid keys on the same curve, $Z_A = Z_B = Z$. HKDF-SHA-256 derives purpose-separated material:

$$K_{enc} = \text{HKDF}(Z, \text{“encryption”}), \quad (2.3)$$

$$K_{mac} = \text{HKDF}(Z, \text{“authentication”}), \quad (2.4)$$

$$K_{ctx} = \text{HKDF}(Z, \text{“session-context”}). \quad (2.5)$$

The context should include both identities, protocol version, and a transcript hash to prevent unknown-key-share and cross-protocol confusion.

2.2.3 Message protection

For every message, the sender generates a fresh 96-bit nonce *N* and applies AES-256-GCM:

$$(C, T) = \text{AES-GCM.Enc}(K_{enc}, N, P, A), \quad (2.6)$$

where *P* is plaintext, *A* is authenticated metadata, *C* is ciphertext, and *T* is the GCM authentication tag. The receiver validates the session, nonce, metadata, and tag before releasing plaintext. GCM nonce reuse with the same key is dangerous; uniqueness must therefore be enforced, not merely expected [3].

The manual also describes a separate HMAC-SHA-256 value. AES-GCM already authenticates ciphertext and associated data, so an additional HMAC is not required for ordinary AEAD use. If retained, its key must be independently derived and the protocol must define an unambiguous encrypt-then-MAC encoding.

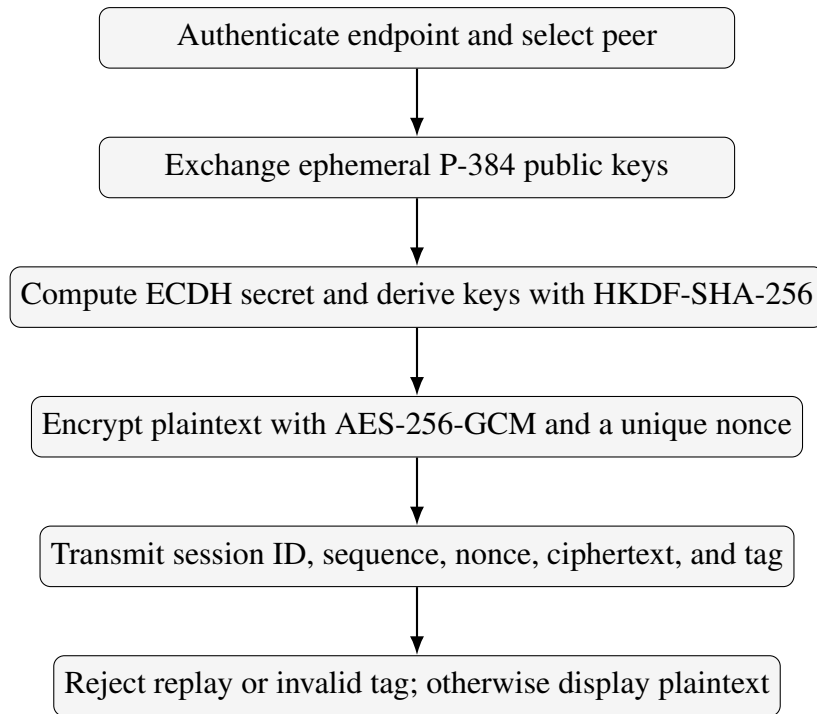


Figure 2.2: Secure-session and message workflow

2.3 User Interface Design

2.3.1 Identity and Nodes views

The Identity view collects a unique handle and initiates key-pair generation and authentication. The Nodes view presents peer cards with status, latency, uptime, protocol version, and a *Secure Link* action. A network radar animation provides activity feedback, while the actual security decision remains based on authenticated protocol state rather than animation.

2.3.2 Session and Vault views

The Session view contains a verification banner, message history, message composer, timestamps, sender labels, signatures, and verification marks. The Vault view demonstrates sealed or expired payloads, key rotation, and self-destruct controls. Since the documented design also states that messages are not persisted, a production specification must clarify whether Vault records are local encrypted objects, synthetic demonstrations, or actual retained ciphertext.

2.3.3 Visual system

The interface uses a dark cyberpunk-inspired palette, glass panels, neon-green active states, soft-red error states, Geist typography, JetBrains Mono for technical values, and responsive navigation. Motion effects include radar sweeps, scanner rings, pulse indi-

cators, and a ciphertext marquee. Accessibility testing should ensure sufficient contrast, visible keyboard focus, reduced-motion support, and non-color status cues.

2.4 Implementation Procedure

2.4.1 Environment and startup

The documented setup requires Python, the project dependencies, and a modern browser. The normal web workflow is:

Listing 2.1: Starting the CIPHERLINK web service

```
1 pip install -r requirements.txt
2 python encryp.io/web_server.py --test
3 python encryp.io/web_server.py
4 # Open http://localhost:5000
```

Two CLI users can be started in separate terminals:

Listing 2.2: Starting two command-line clients

```
1 python encryp.io/client.py alice
2 python encryp.io/client.py bob
```

2.4.2 Connection state machine

A robust implementation should model explicit states: DISCONNECTED, AUTHENTICATING, AVAILABLE, NEGOTIATING, SECURE, and FAILED. Messages must be accepted only in SECURE. Disconnect, authentication failure, malformed key, or tag failure should clear session material and move the connection to a safe state.

2.4.3 WebSocket message structure

The manual describes JSON messages carrying a type, sender, recipient, signature, nonce, and ciphertext. The plaintext field shown in the illustrative format must not be sent for encrypted messages. A safer logical envelope is:

Listing 2.3: Recommended logical encrypted-message envelope

```
1 {
2   "type": "encrypted_message",
3   "version": 1,
4   "session_id": "opaque-session-identifier",
5   "sender": "alice",
6   "recipient": "bob",
7   "sequence": 42,
8   "nonce": "base64-encoded-96-bit-value",
9   "ciphertext": "base64-encoded-ciphertext-and-gcm-tag"
10 }
```

Canonical encoding or a binary format should be specified so that both endpoints authenticate exactly the same metadata bytes.

2.5 Algorithms

Algorithm 1: Authenticated session establishment

Input: Local identity I_A , peer identity I_B , authenticated peer public key Q_B

Output: Session identifier sid and encryption key K_{enc}

- 1 Generate ephemeral P-384 key pair (d_A, Q_A) using a CSPRNG
 - 2 Validate Q_B and its binding to I_B
 - 3 Compute $Z \leftarrow \text{ECDH}(d_A, Q_B)$
 - 4 Compute transcript $h \leftarrow H(I_A \| I_B \| Q_A \| Q_B \| \text{version})$
 - 5 Derive $(K_{enc}, K_{ctx}) \leftarrow \text{HKDF-SHA256}(Z, h)$
 - 6 Compute $sid \leftarrow H(h \| K_{ctx})$ and perform key confirmation
 - 7 Erase d_A and Z when no longer needed
-

Algorithm 2: Message encryption and transmission

Input: Plaintext P , session sid , sequence s , key K_{enc}

Output: Encrypted envelope E

- 1 Verify session state is SECURE
 - 2 Generate a unique 96-bit nonce N
 - 3 Set associated data $A \leftarrow \text{encode}(sid, s, \text{sender}, \text{recipient}, \text{version})$
 - 4 Compute $(C, T) \leftarrow \text{AES-256-GCM.Enc}(K_{enc}, N, P, A)$
 - 5 Set $E \leftarrow (A, N, C, T)$ and transmit E
 - 6 Record s and N for the lifetime of the session
-

Algorithm 3: Message authentication, decryption, and replay checking

Input: Envelope $E = (A, N, C, T)$ and session key K_{enc}

Output: Plaintext P or rejection

- 1 Parse and validate version, identities, session identifier, and lengths
 - 2 **if** *sequence is stale or nonce was previously accepted* **then**
 - 3 reject as replay
 - 4 $P \leftarrow \text{AES-256-GCM.Dec}(K_{enc}, N, C, A, T)$
 - 5 **if** *tag verification fails* **then**
 - 6 erase temporary data and reject
 - 7 Record sequence and nonce, then display P
 - 8 Discard plaintext according to the volatile-retention policy
-

2.6 Security Design Analysis

2.6.1 Eavesdropping and tampering

Captured ciphertext should not reveal plaintext without the session key. AES-GCM also detects unauthorized changes to ciphertext or authenticated metadata. These properties depend on secure key generation, nonce uniqueness, correct tag verification, and endpoint safety; the algorithm name alone is not sufficient [3].

2.6.2 Man-in-the-middle attacks

Unauthenticated Diffie–Hellman is vulnerable to interception. CIPHERLINK therefore relies on certificate-authenticated TLS, identity-key binding, and authenticated key exchange. Certificate pinning can reduce reliance on external trust anchors, but operational procedures are required for secure pin updates and recovery. A visible safety number or QR comparison between peers would provide stronger end-to-end identity verification.

2.6.3 Replay and key compromise

The random nonce required by GCM is not automatically a replay detector. The receiver must track a monotonic sequence number or previously accepted identifiers within each session. Ephemeral ECDH can protect past sessions from later compromise of a long-term authentication key, provided ephemeral private keys and derived secrets are erased and were not generated deterministically.

2.6.4 Quantum-security qualification

P-384 offers a high classical security margin, but elliptic-curve discrete logarithms are vulnerable to Shor’s quantum algorithm [6]. Grover’s algorithm affects exhaustive search against symmetric keys, approximately reducing the effective brute-force exponent; it does not make elliptic-curve key exchange post-quantum secure [7]. Accordingly, CIPHERLINK should be described as crypto-agile and preparing for quantum-resistant migration, not presently quantum-resistant. A future hybrid construction can combine classical ECDH with a standardized post-quantum key-encapsulation mechanism so that the session remains secure if at least one component is secure.

Chapter 3

Performance Evaluation

3.1 Evaluation Environment and Procedure

3.1.1 Documented environment

The documented prototype targets Windows, Linux, and macOS; modern Chromium, Firefox, Safari, and Edge browsers; and Python 3.8 or later. The service uses port 5000 by default, JSON over WebSocket, a configurable 16 KB message limit, 25-second keepalive intervals, and exponential-backoff reconnection [1].

3.1.2 Functional test procedure

The following acceptance procedure evaluates the main workflow without claiming a formal security proof:

1. Run the built-in server and cryptographic self-test.
2. Start two clean clients and create distinct identities.
3. Confirm mutual discovery and establish a secure session.
4. Exchange messages in both directions and compare sent and received plaintext.
5. Modify one ciphertext byte and verify rejection before display.
6. Replay a previously accepted envelope and verify duplicate rejection.
7. Disconnect and reconnect, then confirm that the new session identifier and key material differ.
8. Stop the server and confirm that clients report loss of signaling without presenting an insecure session as active.

3.1.3 Performance measurement procedure

For repeatable benchmarking, at least 1,000 iterations should be executed after warm-up on a named CPU and operating system. Measurements should report median, 95th percentile, and standard deviation for key generation, ECDH, HKDF, 1 KB AES-GCM encryption, decryption, and end-to-end message latency. Network latency should be separated from cryptographic processing time.

3.2 Results Analysis and Testing

3.2.1 Functional results from project documentation

The supplied manual reports successful creation of the web server, identity generation, peer registration, connection request and acceptance, ECDH completion, AES session establishment, encrypted transmission, and decrypted display. It also describes expected rejection behavior for invalid certificates, modified authentication tags, and duplicate nonces. Because source code, automated-test output, and packet captures were not supplied with the report materials, these are documented prototype outcomes rather than independently reproduced measurements.

Table 3.1: Functional verification matrix

Test	Expected secure behavior	Evidence status
Identity creation	Generate P-384 key pair and register authenticated handle	Described in manual
Peer connection	Request, accept, exchange keys, enter secure state	Described in manual
Round-trip message	Receiver outputs exactly the sender's plaintext	Example session supplied
Ciphertext modification	Authentication fails; plaintext is not released	Defense behavior specified
Replay	Duplicate session sequence/nonce is rejected	Defense behavior specified
Re-keying	New session produces different ephemeral keys	Manual verification procedure
Persistence	Server does not retain readable message history	Design claim; audit required

3.2.2 Reported performance targets

Table 3.2 reproduces the approximate ranges in the project documentation. They are plausible design targets for a modern CPU, not controlled measurements from the current report environment.

These values suggest that network and UI delays will normally dominate per-message cryptographic cost. Session setup is more expensive than encrypting one short message,

Table 3.2: Manual-reported approximate cryptographic operation times

Operation	Approximate time
ECDH key agreement	0.5–2 ms
AES-256-GCM encryption of 1 KB	0.05–0.2 ms
HMAC-SHA-256 over 1 KB	0.02–0.1 ms
Full message encryption/decryption	0.1–0.5 ms
P-384 key generation	50–200 ms

but it occurs infrequently. An optimized implementation should therefore prioritize connection reliability, correct state transitions, and batch-safe UI rendering rather than weakening cryptographic parameters for marginal speed gains.

3.2.3 Threat-resistance results

Table 3.3 summarizes the relationship between threats and controls. The result is conditional: each defense works only if its supporting assumptions hold.

Table 3.3: Threats, controls, and remaining risk

Threat	Primary control	Remaining risk
Passive capture	TLS for signaling; AES-GCM at endpoints	Metadata, endpoint compromise, traffic analysis
Message alteration	GCM authentication tag and authenticated metadata	Implementation errors or nonce reuse
MITM	Certificates, pinning, authenticated key binding	Compromised CA/server or unsafe pin rotation
Replay	Session ID and receiver-side sequence/nonce tracking	State loss, race conditions, unbounded cache
Long-term key theft	Ephemeral ECDH and key erasure	Live-session malware and memory extraction
Server breach	End-to-end content encryption and no plaintext storage	Peer directory and traffic metadata exposure
Quantum adversary	AES-256 retains a large symmetric margin	P-384 ECDH is not post-quantum secure
Denial of service	Rate and resource limits, backoff	Distributed attacks and server dependency

3.3 Overall Discussion

The prototype integrates multiple security layers into a coherent user workflow. Its strongest architectural decision is to perform content protection at endpoints rather than relying exclusively on transport encryption. The design also recognizes the value of ephemeral sessions, authenticated encryption, explicit peer status, and limited storage.

The largest technical risk is ambiguity between product language and cryptographic guarantees. The phrases “unique symmetric key per message,” “fresh key pair per session,” and “AES-GCM plus HMAC” describe different possible constructions and must be made consistent in a normative protocol specification. Likewise, true peer-to-peer transport, server-relayed ciphertext, and WebSocket signaling have different metadata and availability properties. Precise message schemas, state transitions, transcript binding, error handling, and key lifetimes are necessary before an external security review.

3.3.1 Complex engineering problem discussion

The project demonstrates interdependence between cryptography, networking, interface design, and operations. For example, an animation may show a secure link while the WebSocket is reconnecting; unless UI state is derived from authenticated protocol state, a presentation defect becomes a security defect. Similarly, a correct encryption function cannot prevent replay without session state, and a strong key exchange cannot authenticate a peer without trustworthy identity binding. The engineering challenge is therefore the secure composition of components, not simply selection of strong algorithms.

Chapter 4

Conclusion

4.1 Discussion

CIPHERLINK demonstrates a practical architecture for real-time, endpoint-protected messaging using Python, browser technologies, WebSockets, authenticated identity exchange, P-384 ECDH, HKDF-SHA-256, and AES-256-GCM. The web and CLI workflows make peer discovery, connection establishment, encryption, and verification visible to users. Based on the supplied documentation, the design can provide confidentiality, integrity, replay resistance, and forward secrecy under its stated assumptions. The analysis also shows that these guarantees depend on precise protocol composition, authenticated key binding, unique nonces, receiver-side replay state, secure deletion, and honest representation of the system’s trust boundaries.

4.2 Limitations

The principal limitation is that P-384 ECDH is not resistant to a cryptographically relevant quantum computer. The project also lacks, in the supplied materials, independently reproducible benchmark logs, source-level audit evidence, formal protocol verification, production-scale load testing, and usability or accessibility studies. Certificate lifecycle management, endpoint malware, metadata leakage, account recovery, group messaging, NAT traversal, and distributed denial of service remain outside the demonstrated scope. Python garbage collection also makes reliable zeroization of all immutable secret objects difficult, so claims that data are “immediately destroyed from memory” should be qualified.

4.3 Scope of Future Work

Future development should:

- replace or augment P-384 ECDH with a standardized post-quantum hybrid key-establishment design;

- publish a versioned protocol specification with canonical encodings, transcript binding, key confirmation, sequence rules, and error behavior;
- add peer-verifiable safety numbers or QR codes independent of the signaling server;
- use operating-system-backed secure key storage and document certificate rotation and recovery;
- create automated unit, integration, fuzz, replay, malformed-input, concurrency, and load tests;
- conduct independent cryptographic review and packet-level verification;
- measure performance on named devices with reproducible scripts and statistical summaries; and
- add accessibility controls, reduced-motion support, and user studies of security indicators.

References

- [1] CIPHERLINK Project Team. *CIPHERLINK: User Manual and Technical Documentation*, version 1.0 edition, 2026. Project documentation supplied with the laboratory project.
- [2] Elaine Barker, Lily Chen, Allen Roginsky, Apostol Vassilev, and Richard Davis. Recommendation for pair-wise key-establishment schemes using discrete logarithm cryptography. Technical Report NIST Special Publication 800-56A Revision 3, National Institute of Standards and Technology, 2018.
- [3] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/counter mode (GCM) and GMAC. Technical Report NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007.
- [4] Hugo Krawczyk and Pasi Eronen. HMAC-based extract-and-expand key derivation function (HKDF). RFC 5869, 2010.
- [5] Eric Rescorla. The transport layer security (TLS) protocol version 1.3. RFC 8446, 2018.
- [6] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pages 124–134, 1994.
- [7] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996.